Software Requirements Specification (SRS) for

SportStats West Notification Engine

Prepared By: Tyler Nguyen Vo

Organization: Sportstats West

Date: Aug 14, 2026 (Rev. 4)

Revision Notes (Rev. 3 -> Rev. 4)

- SendGrid replaced with Resend as the email delivery provider (FR-3.1). SendGrid's permanent free tier was discontinued industry-wide in 2025; Resend was selected over Mailjet for simpler, transactional-focused API ergonomics, accepting a lower free-tier daily cap (100/day vs Mailjet's 200/day) as a tradeoff. See updated Section 3.1.3 and Section 5.
- Column and field naming standardized on "bib" (integer) across all tables (Participants, Subscribers, Readings, Notifications), matching RaceResult 14's native field name and type, superseding the earlier "bid" (string) naming used in Rev. 2/3. See updated Section 6.

Revision Notes (Rev. 2 -> Rev. 3)

- Ingestion listener and admin REST API confirmed as two separate standalone processes/scripts, not one process sharing an event loop. See updated Section 2.1.
- FR-1.4 dedupe logic specified as write-then-read: every reading is inserted unconditionally for audit purposes, followed by a read-only query for other readings on the same bid within the window before a notification is triggered. See updated Section 3.1.1.
- Added technical constraint requiring SQLite WAL mode, since two separate processes now read/write the same database file concurrently. See updated Section 4.
- Added composite index requirement on Readings(bid, timestamp) to support the dedupe query at scale. See updated Section 6.

1. Introduction

1.1 Purpose

This document specifies the requirements for the "SportStats Notification Engine." This tool is designed to provide real-time email updates to interested parties (e.g., race directors, parents, or sweep teams) when an athlete crosses a RaceResult decoder. SMS delivery was considered and deferred; see Section 3.1.3 for the current communication protocol scope.

1.2 Scope

- Project Identification: SportStats Notification Engine.
- Project Goals: To provide a lightweight tracking solution for smaller races that lack the infrastructure for comprehensive live tracking.
- Target Users: Race Directors (tracking first male/female, sweep), Parents/Guardians (child tracking), and Event Staff.
- Inclusions: Real-time data ingestion from RaceResult 14, automated notification triggering, and a subscriber mapping database.
- Exclusions: This system is not a public results portal or a high-latency statistical analysis tool. SMS delivery is out of scope for this revision.

1.3 Definitions and Abbreviations

- BID: User bib ID.
- RaceResult 14: The timing software that exports bib crossing data.
- Interested Party: Any individual registered to receive a notification for a specific BID.

2. Overall Description

2.1 Product Perspective

The system comprises three named components, deployed as two independent processes on a single venue laptop:

- Ingestion listener (Process 1): a standalone Python script binding a raw TCP socket on port 3099, listening for an inbound connection from RaceResult 14. RR14 acts as the client and dials in; the listener acts as the server. Runs on a thread-per-connection model.
- Sync+dispatch worker: polls the Sheets API every 20 seconds and sends notifications via SendGrid. Runs as part of, or alongside, the ingestion listener process.
- Admin REST API (Process 2): a separate standalone FastAPI/uvicorn process, LAN-only, providing read access to logs and counts.

The two processes do not communicate directly and share no in-memory state. They coordinate solely through the shared SQLite database file. This was a deliberate choice for crash isolation: a fault in the ingestion listener does not take down the admin API, and vice versa. It also means log/count data read via the admin API is not guaranteed to reflect the very latest ingestion event in real time; a short lag is acceptable, per product decision.

2.2 Product Functions

- Ingest live BIB crossing data (BibID) from RaceResult 14 via specified ports.
- Map received BibIDs to a subscriber store supporting a 1:N relationship (one BID to multiple contacts).
- Dispatch automated email notifications via SendGrid to all mapped subscribers.
- Log all sent notifications for audit purposes.

2.3 User Characteristics

- Administrators: Sportstats staff responsible for setting up the local ingestion port, pre-loading the Participants list, configuring the subscription Google Form, and provisioning Sheets API / SendGrid credentials.
- Subscribers: End-users receiving email alerts via the subscription Google Form.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Data Integration

ID	Requirement	Notes
FR-1.1	The system SHALL connect to RaceResult 14 via localhost on a specified port to receive real-time bib scan data.	RR14 connects as client; listener binds and listens as server on port 3099.
FR-1.2	The system SHALL poll the Google Form's linked Google Sheet via the Sheets API every 20 seconds to ingest new subscriptions, merging new rows into the subscriber store without replacing existing data.	Supersedes prior FR-1.2/FR-1.3, which duplicated this requirement with conflicting intervals (20s vs. 15-30s).
FR-1.3	The BID field in the subscription Google Form SHALL be a dropdown menu populated by the pre-loaded Participants list.	UI Mapping
FR-1.4	The system SHALL ignore duplicate BID reads within a 60-second window to prevent redundant notifications. Every raw reading is written to the Readings table unconditionally for audit purposes; deduplication is enforced by a subsequent read-only query for other Readings rows on the same bib within the window, checked before a notification is triggered.	Deduplication. Write-then-read pattern; DB-backed so the window survives a listener process restart.

3.1.2 Notification Logic

ID	Requirement	Notes
FR-2.1	Upon a valid BibID read that is not flagged as a duplicate per FR-1.4, the system SHALL trigger notifications to all mapped subscribers for that specific BID within 5 seconds.	Timing
FR-2.2	The system SHALL support customizable message templates (e.g., "Athlete [Name] has just crossed the Finish Line!"), where [Name] is resolved from the Participants table.	Customization

3.1.3 Communication Protocols

ID	Requirement	Notes
FR-3.1	The system SHALL integrate with the Resend free tier API to deliver all electronic mail alerts.	Provider. Supersedes SendGrid (Rev. 2/3); SendGrid discontinued its permanent free tier industry-wide in 2025. Resend free tier: 3,000 emails/month, 100/day cap, 1 domain.

3.2 Non-Functional Requirements

ID	Requirement	Notes
NFR-1	Latency: Notifications SHALL be dispatched within 5 seconds of the decoder read ingestion.	Performance
NFR-2	Reliability: The system SHALL maintain a local queue to retry notification delivery, attempting up to 3 retries with a 10-second backoff interval between attempts.	Fault Tolerance
NFR-3	Scalability: The architecture SHALL support concurrent notification bursts of up to 200 messages per minute.	Throughput
NFR-4	Security: Subscriber contact information SHALL be encrypted at rest using SQLCipher.	Data Privacy

4. Technical Constraints

- Dependencies: Stable internet connection at the finish line or timing station to reach external communication APIs (Sheets API, SendGrid).
- Input Format: RaceResult 14 (RR14) must be configured to export raw BID data to localhost on port 3099, as tab-delimited JSON pulses.
- Deployment: Runs on a laptop at the venue on the local network as two separate processes (ingestion listener, admin REST API). The admin REST API is LAN-only and is not exposed to the public internet.
- Concurrency: Since the listener and admin API are separate processes accessing the same SQLite file, the database SHALL run in WAL (Write-Ahead Logging) journal mode to allow concurrent reads and writes without lock contention. Each process SHALL use its own database connection/session; sessions are not shared across threads or processes.

5. Technical Stack

Python, FastAPI, SQLAlchemy, SQLite, google-api-python-client, resend.

6. Database Schema

Single-race scoped (no race_id). Rebuilt/reset per event.

Participants

Column	Description
id	Primary key.
bib	Bib ID, unique. Source of truth for the Form dropdown (FR-1.3) and template [Name] resolution (FR-2.2).
name	Participant's full name.

Subscribers

Column	Description
id	Primary key.
bib	Foreign key -> Participants.bib.
name	Subscriber's own name (the interested party, not the athlete).
email	Encrypted at rest per NFR-4.
subscribed_at	Timestamp the row was first synced from the Sheet. Unique constraint on (bib, email) so re-polling the Sheet is insert-or-ignore, not a duplicate insert.

Readings

Column	Description

id	Primary key.
bib	Bib ID read from the decoder.
timestamp	When the read was received, used for the FR-1.4 60-second dedupe window.

A composite index on (bib, timestamp) SHALL be created on this table. The FR-1.4 dedupe check queries by bib and a timestamp range on every single reading; without this index, the query degrades to a full table scan as the table grows over the course of a race.

Notifications

Column	Description
id	Primary key.
bib	Bib ID that triggered the notification, for audit traceability back to the Readings row.
subscriber_id	Foreign key -> Subscribers.id.
status	pending / sent / failed.
attempts	Count of delivery attempts so far, checked against NFR-2's 3-retry cap.
last_attempt_at	Timestamp of the most recent attempt, used to enforce the 10-second backoff.
sent_at	Timestamp of successful delivery. Null until status = sent.
error_message	Last error returned by SendGrid, if any. Supports debugging failed sends.

Sync_State

Column	Description
id	Primary key (single row).
last_row_synced	Row marker/timestamp from the last successful Sheets poll, so FR-1.2 doesn't reprocess the full Sheet every 20 seconds.

Logs

Column	Description
id	Primary key.
message	System event message.
severity	info / warn / error.
timestamp	When the event occurred.